

Arrays and Hashing Masterclass: Day 2

Advanced Problem Solving Patterns

Presented by: QMML Society

Encode and Decode Strings

Problem: Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network to another machine and decoded back to the original list of strings.

Constraints:

- `0 <= strs.length < 100` (List can be empty)
- `0 <= strs[i].length < 200`
- `strs[i]` contains any possible character out of 256 valid ASCII characters.

2

Core Challenge:

Simple delimiters like `,` or `#` will fail because those exact characters might legitimately appear within the strings themselves!

Encode and Decode: Length-Prefixing Strategy

Concept:

Instead of relying purely on a delimiter character, we prepend the exact **length** of the string followed by a known delimiter.

Format: `<length>#<string>`

- **Example:** `["Hello", "World"]`
- "Hello" (length 5) becomes `5#Hello`
- "World" (length 5) becomes `5#World`
- Encoded string: `5#Hello5#World`

3

Why this is robust:

When decoding, we read the number until `#`. This tells us EXACTLY how many characters to jump forward for the string content. Any extra `#` characters inside the string are harmlessly treated as regular content!

Encode and Decode Strings: Algorithm (Pseudocode)

```
FUNCTION encode(list strs):
    CREATE empty string 'encoded_res'
    FOR EACH 'string' IN strs:
        length = length of 'string'
        APPEND (length + '#' + 'string') TO 'encoded_res'
    RETURN 'encoded_res'

FUNCTION decode(string s):
    CREATE empty list 'decoded_res'
    pointer i = 0
    WHILE i < length of s:
        delimiter_index = FIND index of next '#' starting from i
        length = INTEGER( substring from i to delimiter_index )

        start_of_string = delimiter_index + 1
        end_of_string = start_of_string + length

        extracted_string = substring from start_of_string to end_of_string
        APPEND extracted_string TO 'decoded_res'

        i = end_of_string // Move pointer forward

    RETURN 'decoded_res'
```

Encode and Decode Strings: Python Implementation

```
class Solution:
    def encode(self, strs: list[str]) -> str:
        encoded_parts = []
        for s in strs:
            # Append length as string + delimiter + actual string
            encoded_parts.append(str(len(s)))
            encoded_parts.append('#')
            encoded_parts.append(s)

        # Join into single string for fast concatenation
        return "".join(encoded_parts)

    def decode(self, s: str) -> list[str]:
        decoded_strs, i = [], 0

        while i < len(s):
            # Find delimiter to isolate the length integer
            j = s.find('#', i)
            length = int(s[i:j])

            # Extract exactly 'length' characters skipping the '#'
            decoded_strs.append(s[j + 1 : j + 1 + length])

            # Jump pointer completely over the processed chunk
            i = j + 1 + length

        return decoded_strs
```

Encode and Decode Strings: Java Implementation

```
class Solution {  
  
    public String encode(List<String> strs) {  
        StringBuffer sb = new StringBuffer();  
        for(String str : strs){  
            sb.append(str.length()).append('#').append(str);  
        }  
        return sb.toString();  
    }  
  
    public List<String> decode(String str) {  
        List<String> result = new ArrayList<>();  
        int i = 0;  
        while(i < str.length()){  
            int delimiterIndex = str.indexOf('#', i);  
            int length = Integer.parseInt(str.substring(i, delimiterIndex));  
            i = delimiterIndex + 1;  
            String currentString = str.substring(i, i + length);  
            result.add(currentString);  
            i = i + length;  
        }  
        return result;  
    }  
}
```

Products of Array Except Self

Problem: Given integer array `nums`, return an array `output` where `output[i]` is the product of all the elements of `nums` except `nums[i]`.

Constraints & Follow-up:

- Must run in $O(n)$ time.
- **CANNOT** use the division operation. (Normally you would just calculate total product and divide by `nums[i]`).

Example:

- Input: `nums = [1, 2, 4, 6]`
- Output: `[48, 24, 12, 8]`
- `output[0] = 2 * 4 * 6 = 48`
- `output[1] = 1 * 4 * 6 = 24`

Products Except Self: Prefix-Suffix Approach

Concept:

The product of all elements except `nums[i]` is exactly:

`(Product of all elements to the LEFT) * (Product of all elements to the RIGHT)`

Two-Pass Optimization:



1. **Pass 1 (Left to Right):** Store the running product of elements to the *left* of each index in our `output` array.
 2. **Pass 2 (Right to Left):** Keep a running product of elements to the *right* of the index. Multiply it by the existing value in the `output` array.
- **Time Complexity:** $O(n)$
 - **Space Complexity:** $O(1)$ (Excluding output array)

Products Except Self: Algorithm (Pseudocode)

```
FUNCTION productExceptSelf(array nums):  
    n = length of nums  
    CREATE array 'output' of size n, filled with 1s  
  
    // First Pass: Left Products  
    prefix = 1  
    FOR i = 0 TO n - 1:  
        output[i] = prefix  
        prefix = prefix * nums[i]  
  
    // Second Pass: Right Products  
    suffix = 1  
    FOR i = n - 1 DOWN TO 0:  
        // Multiply right product with the stored left product  
        output[i] = output[i] * suffix  
        suffix = suffix * nums[i]  
  
    RETURN output
```

Products Except Self: Python Implementation

```
class Solution:
    def productExceptSelf(self, nums: list[int]) -> list[int]:
        n = len(nums)
        # Initialize output array with 1s
        output = [1] * n

        # Calculate products of elements strictly to the left
        prefix_product = 1
        for i in range(n):
            output[i] = prefix_product
            prefix_product *= nums[i]

        # Calculate products of elements strictly to the right
        # and multiply existing left product
        suffix_product = 1
        for i in range(n - 1, -1, -1):
            output[i] *= suffix_product
            suffix_product *= nums[i]

        return output
```

Products Except Self: Java Implementation

```
class Solution {
    public int[] productExceptSelf(int[] nums) {
        int[] result = new int[nums.length];

        // Initialize result with 1s
        for(int i = 0; i < nums.length; i++){
            result[i] = 1;
        }

        // Prefix pass
        int prefixProduct = 1;
        for(int i = 0; i < nums.length; i++){
            result[i] = prefixProduct;
            prefixProduct *= nums[i];
        }

        // Suffix pass
        int suffixProduct = 1;
        for(int i = nums.length - 1; i >= 0; i--){
            result[i] *= suffixProduct;
            suffixProduct *= nums[i];
        }

        return result;
    }
}
```

Valid Sudoku

Problem: Determine if a 9×9 Sudoku board is valid. Only filled cells need to be validated according to standard Sudoku rules:

1. Each row contains digits 1–9 unique.
2. Each column contains digits 1–9 unique.
3. Each 3×3 sub-box contains digits 1–9 unique.

Core Concept:

We need to track previously seen digits. Hash Sets are perfect for this!

For a cell `(r, c)` :

- Row Check: `digit in rows[r]`
- Col Check: `digit in cols[c]`
- Box Check: How do we map `(r, c)` to a unique 3×3 box index?

Valid Sudoku: Box Indexing Math

A 9×9 board is divided into nine 3×3 grids (boxes).

To map any absolute cell coordinate (r, c) to a specific box index 0–8 :

```
box_index = (r // 3) * 3 + (c // 3)
```

- $r // 3$ gives the "block row" (0, 1, or 2).
- $c // 3$ gives the "block column" (0, 1, or 2).
- Multiplying the block row by 3 shifts us to the correct set of 3 horizontal blocks!

If we maintain an array of 9 Hash Sets (`boxes[9]`), we can identify duplicates in $O(1)$ time in a single pass of the board.

Valid Sudoku: Algorithm (Pseudocode)

```
FUNCTION isValidSudoku(2D_array board):  
    CREATE array of 9 empty Sets for 'rows'  
    CREATE array of 9 empty Sets for 'cols'  
    CREATE array of 9 empty Sets for 'boxes'  
  
    FOR r = 0 TO 8:  
        FOR c = 0 TO 8:  
            digit = board[r][c]  
            IF digit == '.': CONTINUE  
  
            box_idx = (r // 3) * 3 + (c // 3)  
  
            IF digit EXISTS IN rows[r]  
                OR digit EXISTS IN cols[c]  
                OR digit EXISTS IN boxes[box_idx]:  
                RETURN False // Duplicate Found!  
  
            ADD digit TO rows[r]  
            ADD digit TO cols[c]  
            ADD digit TO boxes[box_idx]  
  
    RETURN True
```

Valid Sudoku: Python Implementation

```
class Solution:
    def isValidSudoku(self, board: list[list[str]]) -> bool:
        # Array of Sets for constant time lookups
        rows, cols, boxes = [set() for _ in range(9)], [set() for _ in range(9)], [set() for _ in range(9)]

        for r in range(9):
            for c in range(9):
                digit = board[r][c]
                if digit == '.':
                    continue

                # Formula to map absolute coords to 1 of 9 box indices
                box_idx = (r // 3) * 3 + (c // 3)

                # Check for existing violations in the respective sets
                if digit in rows[r] or digit in cols[c] or digit in boxes[box_idx]:
                    return False

                # Mark as seen
                rows[r].add(digit)
                cols[c].add(digit)
                boxes[box_idx].add(digit)

        return True
```

Valid Sudoku: Java Implementation

```
class Solution {
    public boolean isValidSudoku(char[][] board) {
        Set<Character>[] rows = new HashSet[9];
        Set<Character>[] cols = new HashSet[9];
        Set<Character>[] boxes = new HashSet[9];

        for (int i = 0; i < 9; i++) {
            rows[i] = new HashSet<>(); cols[i] = new HashSet<>(); boxes[i] = new HashSet<>();
        }

        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                char digit = board[r][c];
                if (digit == '.') continue;

                int boxIdx = (r / 3) * 3 + (c / 3);

                // add() returns false if item already exists in the set
                if (!rows[r].add(digit) || !cols[c].add(digit) || !boxes[boxIdx].add(digit)) {
                    return false;
                }
            }
        }
        return true;
    }
}
```


Longest Consecutive Sequence

Problem: Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.

- **Example:** `[100, 4, 200, 1, 3, 2]` -> Longest is `[1, 2, 3, 4]`. Output is `4`.
- **Constraint:** Must run in $O(n)$ time complexity!

Core Concept:

Sorting would take $O(n \log n)$. Instead, dump everything into a **Hash Set** for $O(1)$ lookups.

To prevent doing redundant work, how do we know where a sequence starts?

- A number `num` is ONLY the true start of a sequence if `num - 1` does NOT exist in the set!
- If `num - 1` exists, this number is just in the middle of a sequence. Skip it!

Longest Consecutive Sequence: Algorithm

```
FUNCTION longestConsecutive(array nums):  
    CREATE Set 'num_set' containing all elements from 'nums'  
    longest_streak = 0  
  
    FOR EACH 'num' IN num_set:  
        // Optimization check: Is this the TRUE start of a chain?  
        IF (num - 1) DOES NOT EXIST IN num_set:  
            current_num = num  
            current_streak = 1  
  
            // Loop upwards pushing the sequence forward  
            WHILE (current_num + 1) EXISTS IN num_set:  
                current_num = current_num + 1  
                current_streak = current_streak + 1  
  
            longest_streak = MAXIMUM(longest_streak, current_streak)  
  
    RETURN longest_streak
```

Longest Consecutive Sequence: Python

```
class Solution:
    def longestConsecutive(self, nums: list[int]) -> int:
        # Convert list to Set to enable O(1) existence checks
        num_set = set(nums)
        longest_streak = 0

        for num in num_set:
            # Check if num is the START of a sequence (no preceding number exists)
            if (num - 1) not in num_set:
                current_num = num
                current_streak = 1

                # Check for successive numbers counting upwards
                while (current_num + 1) in num_set:
                    current_num += 1
                    current_streak += 1

                # Store the absolute maximum sequence length found
                longest_streak = max(longest_streak, current_streak)

        return longest_streak
```

Longest Consecutive Sequence: Java

```
import java.util.HashSet;
import java.util.Set;

class Solution {
    public int longestConsecutive(int[] nums) {
        Set<Integer> numSet = new HashSet<>();
        for (int num : nums) numSet.add(num);

        int longestStreak = 0;

        for (int num : numSet) {
            // Check if num is the start of a sequence
            if (!numSet.contains(num - 1)) {
                int currentNum = num;
                int currentStreak = 1;

                // Track upwards consecutive integers
                while (numSet.contains(currentNum + 1)) {
                    currentNum++;
                    currentStreak++;
                }

                longestStreak = Math.max(longestStreak, currentStreak);
            }
        }
        return longestStreak;
    }
}
```

Conclusion: Day 2 Wrap-up

- **Encode/Decode**: Used length-prefixing to create an unambiguous data stream capable of safely capturing any string.
- **Product Except Self**: Demonstrated the power of running Multi-pass algorithms (Prefix/Suffix caching) to reduce massive computational duplication.
- **Valid Sudoku**: Transformed spatial constraints into absolute states using distinct Hash Arrays and Integer division indices.
- **Longest Consecutive**: Used a Hash Set and optimal starting-point logic (`num - 1` missing) to drop worst-case $O(n^2)$ down to pure average $O(n)$ time!